

# Monitoring de systèmes hydrauliques

## Pipeline MLOps — De l'entraînement à la production

Lina Rhiati Hazime    Mohammed Horache    Benjamin Lepourtois    Grégoire Petit



DATA713 — Mars 2026

**Cas d'usage** : Maintenance prédictive industrielle

**Dataset** : UCI — Condition Monitoring of Hydraulic Systems

- 2205 cycles  $\times$  17 capteurs
- 4 états de composants à prédire simultanément
- Labels issus de `profile.txt` (supervisé)

**Formulation** : Classification multi-classe multi-output

→ Pourquoi pas détection d'anomalies ? Données labellisées

→ diagnostic par composant plus actionnable

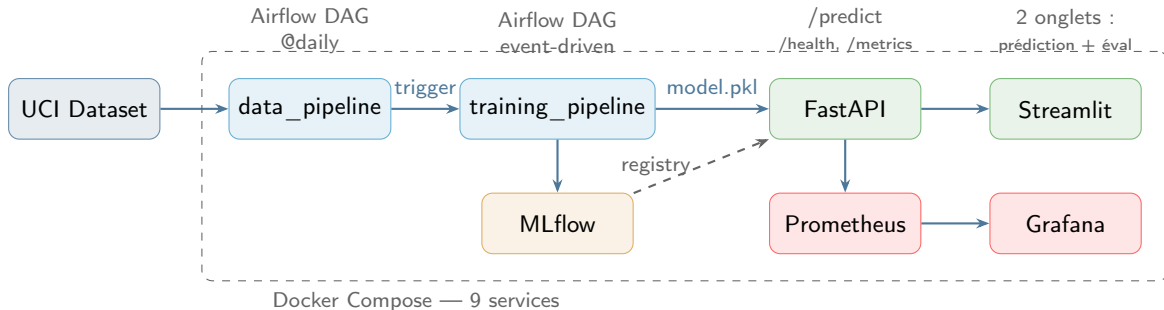
| Cible       | Composant     | Classes |
|-------------|---------------|---------|
| cooler      | Refroidisseur | 3       |
| valve       | Vanne         | 4       |
| pump        | Pompe         | 3       |
| accumulator | Accumulateur  | 4       |

**17 capteurs** :

PS1–PS6, EPS1, FS1–FS2,  
TS1–TS4, VS1, CE, CP, SE

Feature eng. : moyenne par cycle

# Vue d'ensemble de l'architecture



## Pourquoi Docker Compose ?

Un seul docker compose up → 9 services.  
Reproductible, aucune installation manuelle.

## Pourquoi des manifests K8s ?

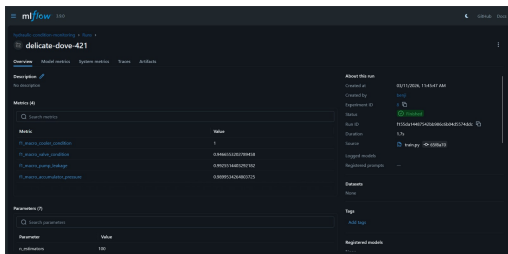
Pipeline CD prêt pour la production. Déploiement conditionnel au secret KUBECONFIG.

# Pipeline ML & Continuous Training

**Modèle :** MultiOutputClassifier(RF)

Un seul entraînement, déploiement plus simple que 4 modèles

**MLflow — Experiment Tracking :**

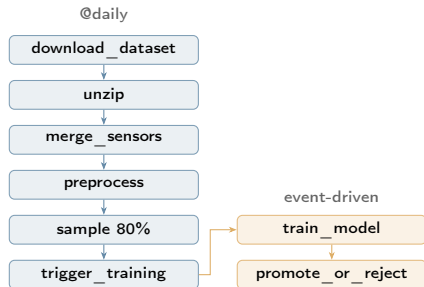


Run MLflow — 4 métriques F1 macro, source train.py

**Choix de conception :**

- F1 macro pour la comparaison (gère le déséquilibre)
- DAG délègue à src/train.py (séparation des resp.)
- Promotion seulement si  $F1_{new} > F1_{prod}$

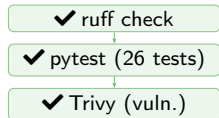
**Logique CT (2 DAGs) :**



Airflow UI — graphe du data\_pipeline (6 tâches)

# Intégration Continue (CI)

## Déclenchée à chaque push & PR



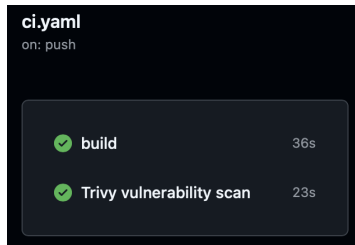
## 26 tests en 3 suites :

| Suite    | #  | Vérifie                    |
|----------|----|----------------------------|
| DAGs     | 9  | structure, task IDs, deps  |
| Model    | 7  | entraînement, F1, features |
| Preproc. | 10 | merge, filtre, NaN         |

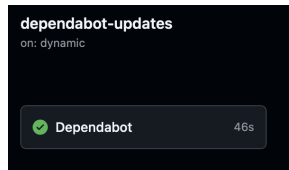
**Outils :** uv + ruff + Trivy + Dependabot

**Workflow :** Feature branches → PR → main

CI gate : merge bloqué si rouge. 30+ PRs.

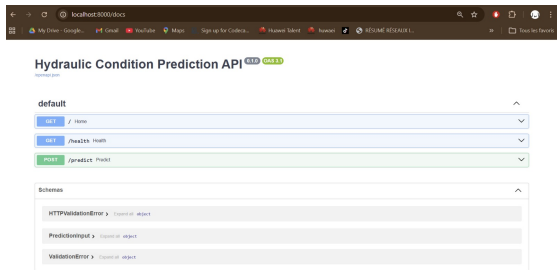


Pipeline CI GitHub Actions — checks verts



Dependabot — mises à jour sécurité automatiques

## FastAPI

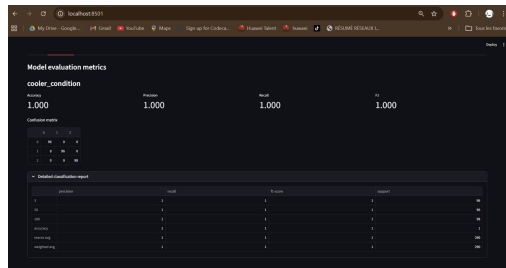


### Swagger UI — 4 endpoints documentés

- POST /predict — 17 capteurs → 4 états
- GET /health, /metrics, /docs

➔ Async, Pydantic, Swagger inclus

## Streamlit — 2 onglets



### Évaluation du modèle — métriques, matrice de confusion

- **Onglet 1** : 17 capteurs → diagnostic 4 composants
- **Onglet 2** : Métriques F1, matrices de confusion

Source : reports/model\_metrics.json

## Docker Compose — Dev / Démo

9 services en un seul docker compose up :

- Airflow (scheduler + webserver + triggerer)
- PostgreSQL (métadonnées Airflow)
- MLflow Tracking Server
- API FastAPI + WebApp Streamlit
- Prometheus + Grafana

→ Reproductibilité totale : zéro install manuelle

### Images Docker optimisées :

Multi-stage build, uv pour les deps,

.dockerignore pour réduire le contexte

## Kubernetes — Production

- Manifests K8s prêts (API + WebApp)
- Déploiement **conditionnel** :  
Job check-deploy vérifie la présence  
du secret KUBECONFIG avant deploy
- Tags images : latest + git SHA  
→ rollback immédiat possible

### Pipeline CD (push sur main) :

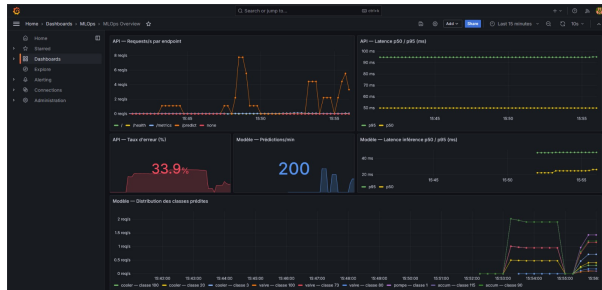


GitHub Actions CD — build, push, check-deploy

Double registry : GHCR + DockerHub

Déploiement K8s conditionnel au secret KUBECONFIG

## Dashboard Grafana



MLOps Overview — req/s, latence p95, error rate, predictions/min

- Dashboard **auto-provisionné** au compose up
- Datasource Prometheus configurée automatiquement

## Prometheus

- Scrape `/metrics` toutes les 15s
- Métriques : requêtes, latence, erreurs HTTP, compteurs de prédictions

## Alerting

- **Airflow callbacks** : email en cas d'échec DAG
- `on_failure_callback` → email avec DAG, task, date
- SMTP configurable dans `airflow.cfg`

→ Bonus barème #12 + #17



# Couverture des objectifs

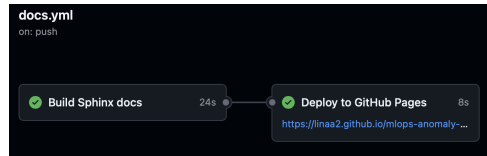
## 10/10 objectifs requis :

| #  | Objectif                               | Statut |
|----|----------------------------------------|--------|
| 1  | Extraction & prétraitement des données | ✓      |
| 2  | Modèle ML                              | ✓      |
| 3  | Model registry (MLflow)                | ✓      |
| 4  | Pipeline Airflow de réentraînement     | ✓      |
| 5  | MLflow experiment tracking             | ✓      |
| 6  | API (FastAPI + Swagger)                | ✓      |
| 7  | WebApp (Streamlit, 2 onglets)          | ✓      |
| 8  | Continuous Training (CT)               | ✓      |
| 9  | Docker + K8s + CI/CD                   | ✓      |
| 10 | Versionnage GitHub & docs              | ✓      |

## Bonus réalisés (3/8) :

| #  | Bonus                         |   |
|----|-------------------------------|---|
| 12 | Monitoring (Prom+Grafana)     | ✓ |
| 14 | Versionnage modèle / rollback | ✓ |
| 17 | Alerting email                | ✓ |
| 11 | Données temps réel            | ✗ |
| 13 | Test de charge (Locust)       | ✗ |
| 15 | Human in the loop             | ✗ |
| 16 | Gestion accès API             | ✗ |

## Documentation auto-générée :



Sphinx — doc technique générée depuis le code